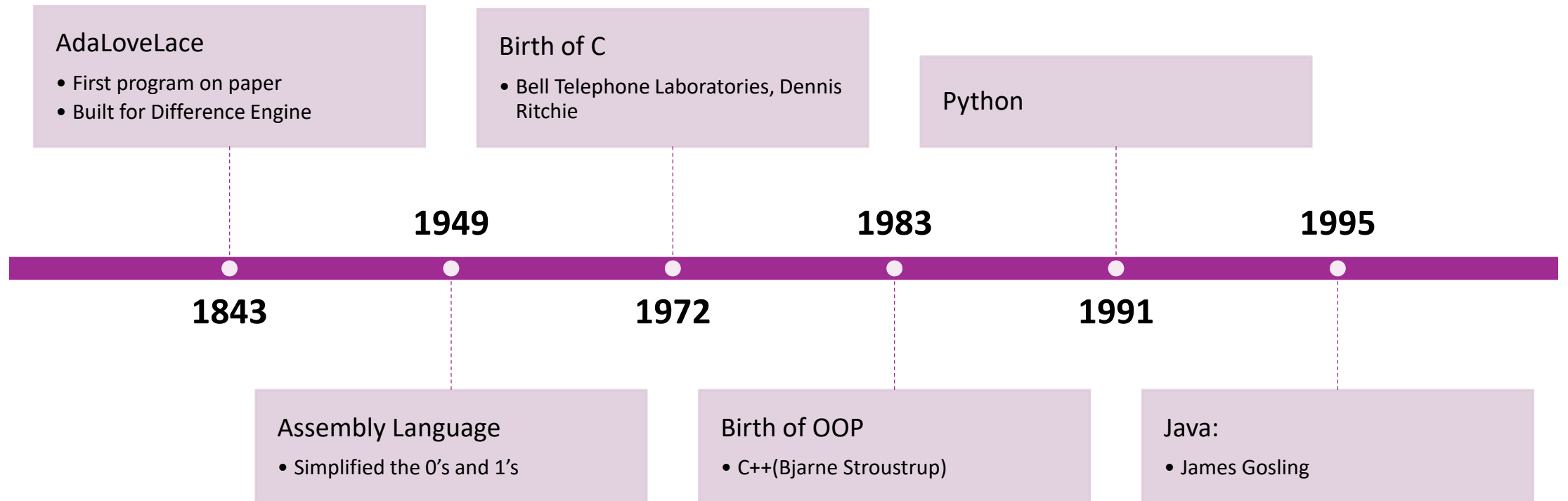


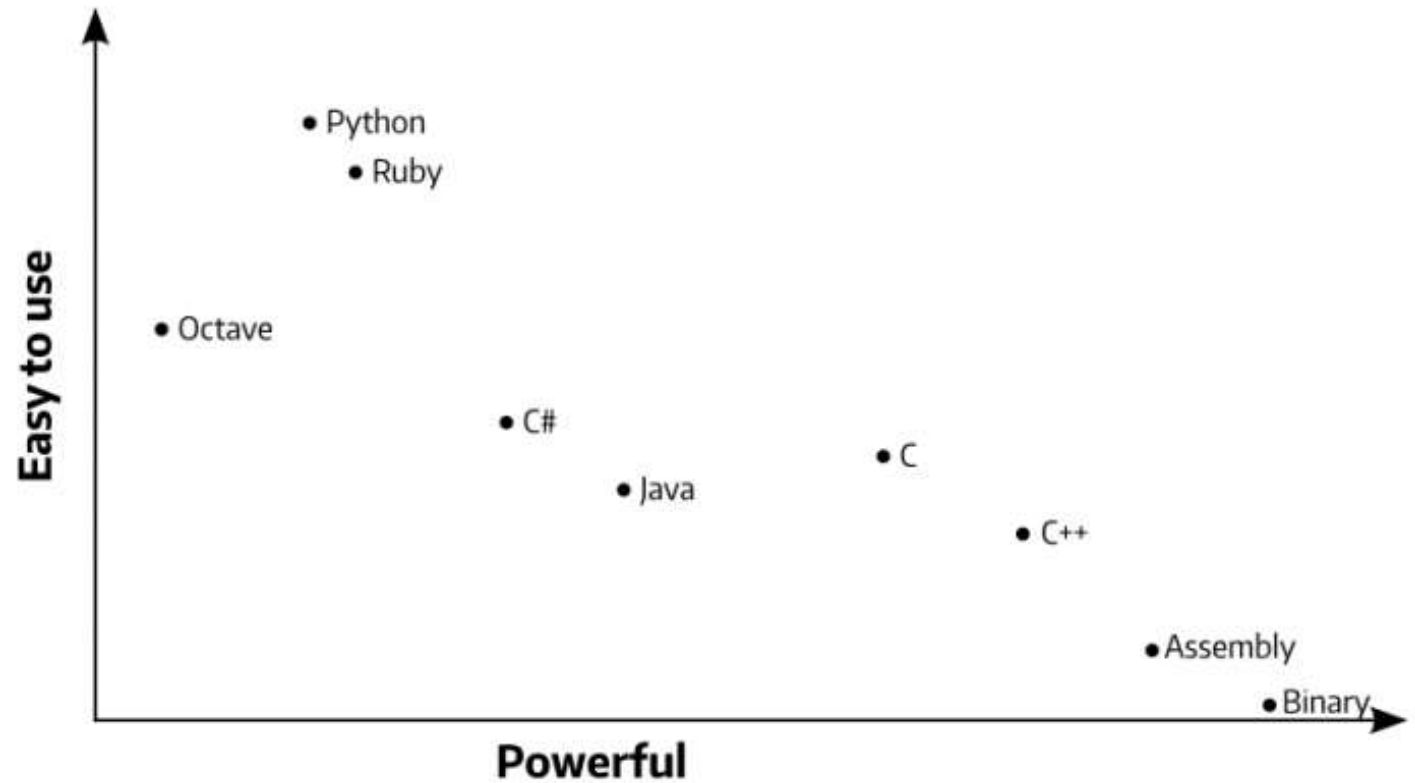
Introduction to Java

By: Bilal Ahmed

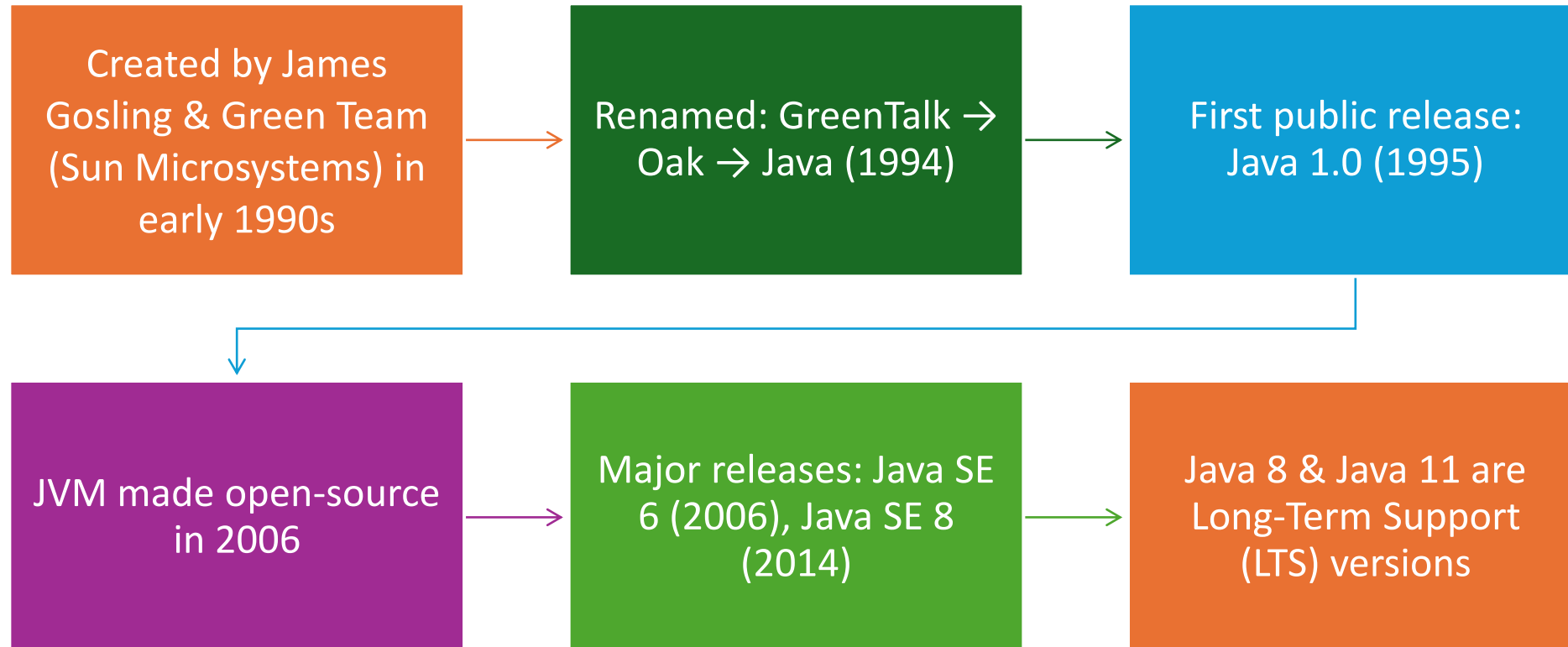
History of Programming



Trade-offs in language design



History of Java



What is Java?



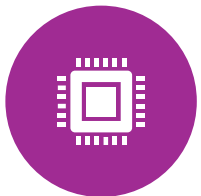
High-level, object-oriented language



Developed by Sun Microsystems (1995), now owned by Oracle



Runs on billions of devices



Platform-independent (Windows, macOS, UNIX)



Used for desktop, mobile, web, and games

Editions of Java



Java Standard Edition (JSE):

Programs for Desktops



Java Enterprise Edition (JEE):

Programs for servers



Java Micro Edition (JME):

Programs for small devices

Types of Java Applications



Standalone Applications



Enterprise Applications



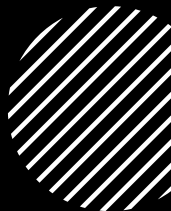
Web Applications



Mobile Applications



Major Companies using Java



Google



Microsoft



Facebook



IBM



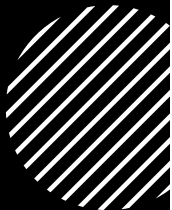
Amazon



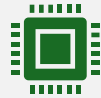
Netflix



Products & Industry Impact



Powering tools/products like JUnit, Eclipse, Hadoop, Android



Widely used by companies (e.g., Google, Accenture)



Adopted by hardware firms like Intel, Symantec, Philips



Official language for Android development & games (e.g., PUBG)



Considered a de facto industry standard since 2007

C++ vs Java

Feature	C++	Java
Programming Type	Procedural + OOP	Object-Oriented
Platform Dependency	Platform-dependent	Platform-independent (WORA)
Compilation	Compiled to machine code	Compiled to bytecode → JVM
Pointer Support	Supported	Not supported
Memory Management	Manual	Automatic (Garbage Collection)
Multiple Inheritance	Supported	Not supported (via classes)
Security	Less secure	More secure
Execution Speed	Faster	Slower than C++

Bytecode —
the magic of
Java (WORA)
A.java
A.class



Bytecode — the
magic of Java —
Write once run
anywhere
(WORA)

Bytecode is an **intermediate, platform-independent code** generated by the Java compiler.

- Java source code → .java
- Compiled by javac
- Output → .class file (**bytecode**)

Bytecode is **not machine code** for Windows, Linux, or macOS.

It is a **universal code** understood by the **Java Virtual Machine (JVM)**.

Why Bytecode Exists

Different computers have different processors and operating systems.

If Java compiled directly to machine code:

- One version for Windows
- One for Linux
- One for macOS

Instead, Java uses **bytecode** as a middle layer.

How Bytecode Enables WORA

- The **same bytecode** runs everywhere
- Only the **JVM changes**, not the program
- This is the **core magic of Java**.

Java **Source Code** (.java)



Java Compiler (javac)



Bytecode (.class)



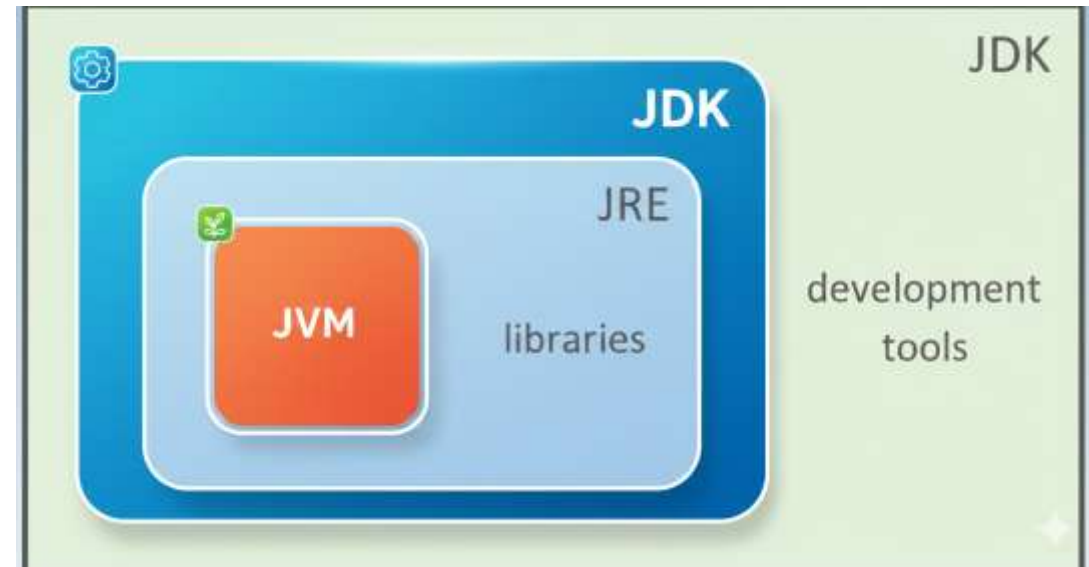
JVM (platform-specific)



Machine **Code**

Java components

- The **JDK** is a **complete package** for **developing and running** Java programs.
- The **JRE** provides the **runtime environment** needed to **run** Java programs.
- The **JVM** is a **virtual machine** that **executes Java bytecode**.



Java Development Kit (JDK)

-
- Complete package for **developing & running** Java programs
 - Includes tools to **compile, debug, and execute** code
 - Contains **JRE + JVM**
 - Platform-dependent (separate installers for OS)
 - Represents the **installed Java version**

Java Runtime Environment (JRE)

-
- Provides environment to **run Java programs**
 - Implementation of **JVM specifications**
 - Contains:
 - JVM
 - Core libraries
 - Java binaries
 - Includes deployment tools & base libraries
 - Does **not** include compiler

Java Virtual Machine (JVM)

-
- Provides runtime environment to execute **bytecode**
 - Converts bytecode → machine code
 - Platform-dependent implementation
 - Can run bytecode from other languages
 - Main functions:
 - Loads code
 - Verifies code
 - Executes code
 - Provides runtime environment

Summary

Component	Meaning
JDK	Develop + Compile
JRE	Run environment
JVM	Execute engine

Just-In-Time (JIT) Compiler

-
- **JIT (Just-In-Time) Compiler** is a component of the **JVM**
 - Converts **bytecode** → **machine code at runtime**
 - Compiles only the **frequently used parts** of the program
 - Improves execution **speed & performance**
 - Works alongside the interpreter inside JVM
 - Key role: Makes Java faster despite being platform-independent

JIT Impact

Without JIT	With JIT
Repeated interpretation	One-time compilation
Slower loops	Faster loops
More CPU overhead	Optimized execution

Code type	Executed by
First-time code	Interpreter
Rarely used code	Interpreter
Frequently used (hot) code	JIT compiled



Analogy

JVM: The machine that does the work.

JRE: The factory environment where the machine operates.

JDK: The factory along with tools to build products.

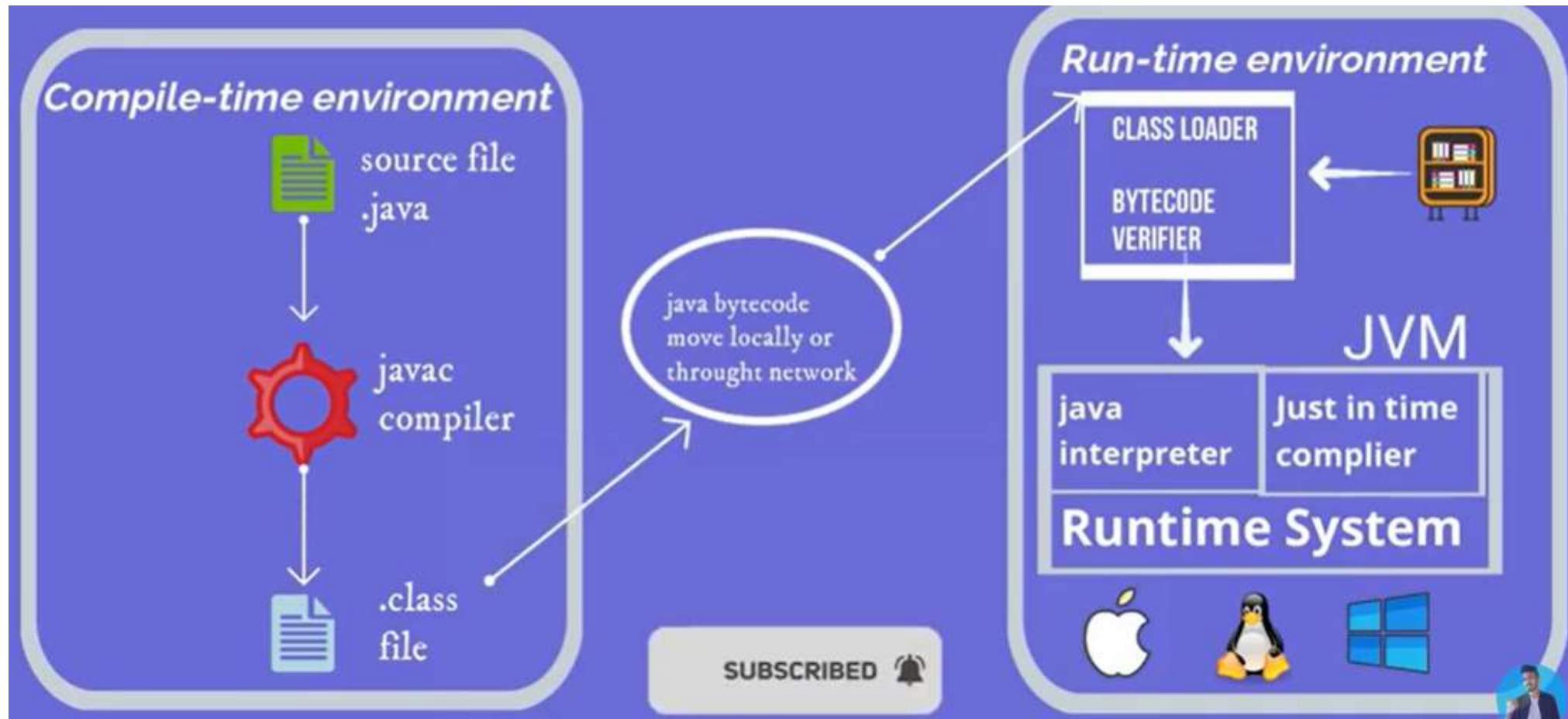
Execution stages

Stage	Tool / Component	Role
1 Writing Code	Developer	Write source code (.java)
2 Compilation	JDK (javac)	Converts .java → Bytecode (.class)
3 Class Loading	JVM	Loads bytecode into memory
4 Bytecode Verification	JVM	Checks security & correctness
5 Interpretation / JIT	JVM	Converts bytecode → Machine code
6 Runtime Execution	JRE + JVM	Provides environment & runs program
7 Output	OS / Hardware	Displays result

Summary

Feature	JDK	JRE	JVM
Purpose	Develop + Run programs	Run programs	Execute bytecode
Contains	JRE + JVM + Tools	JVM + Libraries	Execution engine
Compiler (javac)	✓	✗	✗
Platform Dependency	Yes	Yes	Yes
Main Role	Development	Runtime environment	Code execution

How Java Works?



Java Features

- Simple
 - Easy to understand and use
 - Complex concepts like pointers are eliminated
- Secure:
 - No pointers and memory conflicts
- Portable
 - Bytecode can be carried to any platform
- Object-oriented
 - Every thing is object (Dog, Chair, etc)
- Robust (memory management, exception handling, reliability)
- Architecture-neutral (write once; run anywhere, anytime, forever)
- Interpreted and High Performance (JIT, no compromise on performance)

Classes and Objects

Object:

- Object is real world entity:
- Dog, Chair, Laptop
- Easy to develop programs using real world entities

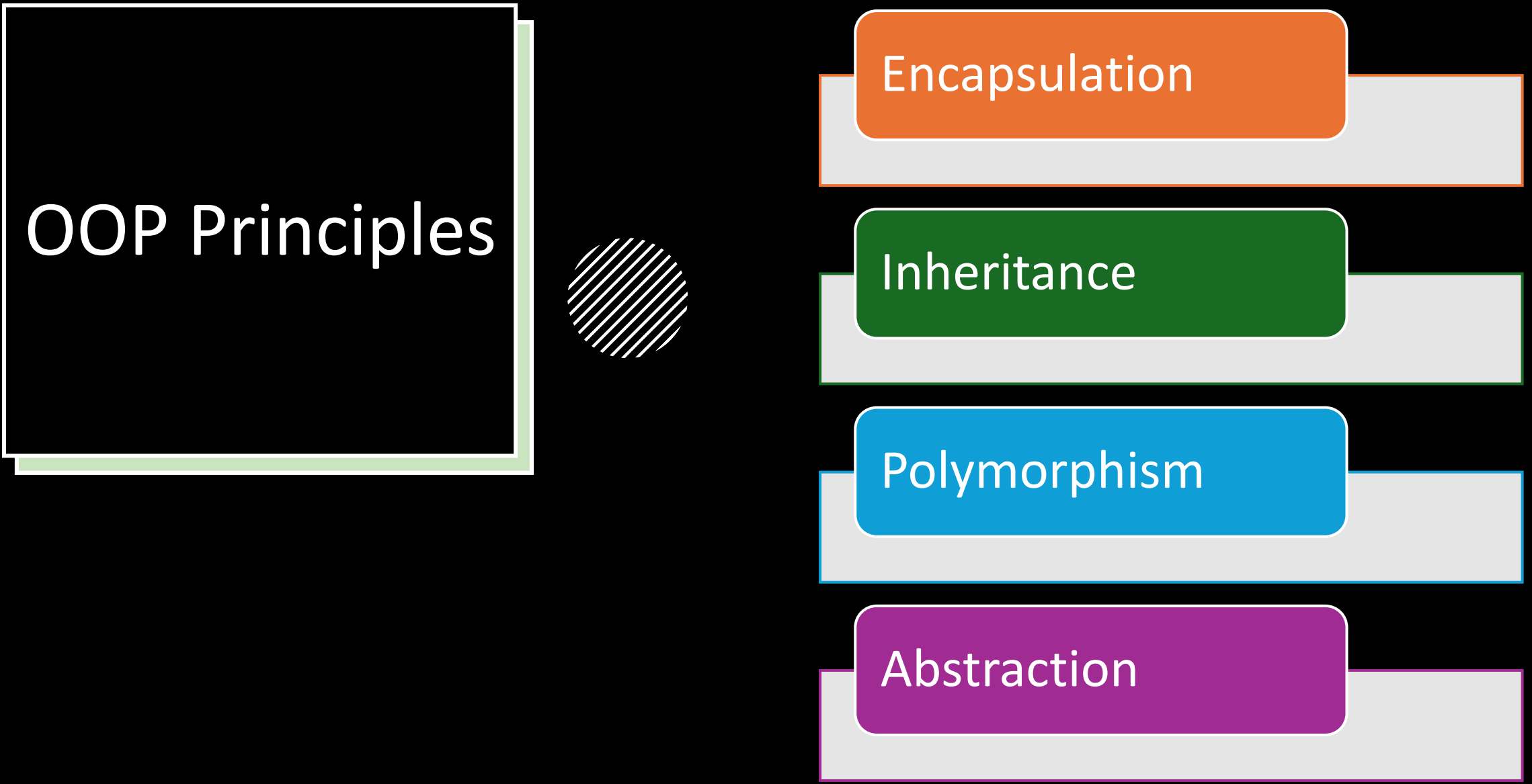
Class:

- Class is a template or blueprint through which objects of different types are created
- Animal, Furniture, Computer

Characteristics of an Object

- An object has state, behavior and identity
 - State:
 - Data values of an object, like legs of a dog
 - Behavior:
 - Functionality of an object like barking of dog
 - Identity:
 - Unique id for each object
 - Used by JVM for unique identification

OOP Principles



Encapsulation

Inheritance

Polymorphism

Abstraction

Java IDEs

IDE	(Pro + Con)
NetBeans	Easy GUI builder 👍 but heavier startup ❌
Eclipse	Highly extensible 👍 but complex UI ❌
IntelliJ IDEA	Smart coding assistance 👍 but paid Ultimate features ❌
BlueJ	Beginner-friendly 👍 but limited for large projects ❌
VS Code	Lightweight + multi-language 👍 but needs extensions for Java ❌



Hello to Java

```
public class first {  
    public static void main(String[]  
args) {  
        System.out.println("Hello from  
java");  
    }  
}
```

Using CMD

```
C:\Users\Asad Computers\Examples>dir
Volume in drive C has no label.
Volume Serial Number is DA49-4422

Directory of C:\Users\Asad Computers\Examples

02/11/2026  12:13 PM    <DIR>          .
02/11/2026  12:11 PM    <DIR>          ..
02/06/2026  06:59 PM                126 first.java
                1 File(s)                126 bytes
                2 Dir(s)  18,034,106,368 bytes free

C:\Users\Asad Computers\Examples>javac first.java

C:\Users\Asad Computers\Examples>dir
Volume in drive C has no label.
Volume Serial Number is DA49-4422

Directory of C:\Users\Asad Computers\Examples

02/11/2026  12:13 PM    <DIR>          .
02/11/2026  12:11 PM    <DIR>          ..
02/11/2026  12:13 PM                419 first.class
02/06/2026  06:59 PM                126 first.java
                2 File(s)                545 bytes
                2 Dir(s)  18,033,885,184 bytes free

C:\Users\Asad Computers\Examples>java first
Hello from Java
```

Bytecode and the class

- .class file is the **compiled bytecode** generated from .java using javac
 - JVM executes bytecode, not source code (.java → .class → JVM → Output)
 - We run using java ClassName because JVM loads **classes automatically**
 - .class extension isn't written since Java execution is **class-based, not file-based**
-

Code Explanation



In Java, **everything is inside a class.**



class → keyword used to define a class



HelloWorld → class name



Must match the file name: first.java



What happens if we remove public next to
class in above code?

Class without public

Class becomes **package-private** (default access)

File name **does not have to match strictly**

Still runnable if it has `main()`

Removing `public` doesn't stop execution

It only restricts class visibility and removes the strict file-name rule.

Simple Analogy



Public class → Like a public website → Anyone can access → Needs exact address.

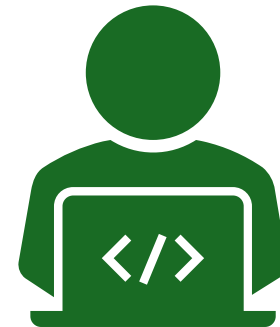


Non-public class → Like an office file → Only internal staff access → Name matching less critical externally.

Method = Function (C++ analogy)



In Java, Method = Function inside
a class



So main() is simply the **entry
function** of Java.

public



JVM must call it from outside the class.



If not public → JVM may not access entry point.



JVM can still run non-public main() in some cases.



public ensures JVM can legally access entry method.

static, (first let's go to C++)



main() is **global**



Not inside any
class



OS calls it
directly



So no static
needed.

Java things



Java rule: Everything must be inside a class.



So main() cannot be global.



It must live inside a class.



Now the problem arises.

That's why static

- **static means:** Method belongs to class itself, not to any object.
- So JVM can call it like: First.main(), no object needed.

C++	Java
main() global	main() inside class
OS calls directly	JVM calls via class
No object issue	Needs static

static



Belongs to **class**, not object.



So JVM can call it **without creating object**.



If not static, an object must be created.



But JVM doesn't create objects automatically.



So `main()` must be static.

void

- Return type = nothing
- main() does not return any value to JVM.
- If we change it to int or string.
- JVM won't accept — signature mismatch.

main

- Method name — fixed entry point.
- JVM always searches for: **main**
- If renamed → program won't start.

String[] args

Command-line arguments array.

Example: `java First Hello 123`

Then:

- `args[0] = "Hello"`
- `args[1] = "123"`

JVM searches specifically for: `main(String[] args)`

Why method name must be main

C++ → OS looks for main

Java → JVM looks for main

If Java allowed any name:

- startProgram()
- run()
- execute()

JVM wouldn't know where to begin.

So designers fixed: Entry method name = main

Why String[] args specifically?

- A standard way to pass command-line inputs to every program.
- **Why array?:** Programs may receive multiple inputs:
- **Like:** `java First file.txt 10 true`
- **Why String?:** Because command-line input is always text initially.
- "10" → parsed to int
- So string is universal container.

Summary

public

- Makes the method accessible to JVM
- JVM must be able to call main() from outside the class

static

- JVM can call main() **without creating an object**
- Program starts before any object exists

main

- Fixed method name
- JVM looks for **exactly this name** to start execution

String[] args

- Used to receive **command-line arguments**
- String[] → array of strings
- args → variable name

Execution Flow

JVM starts

JVM looks for main() method

Execution begins from first statement inside main

Output is printed

System class

System is a predefined (built-in) Java class.

It provides access to system-level resources.

Used for:

- Standard input
- Standard output
- Error output
- System utilities

Belongs to package: `java.lang`

Imported automatically.

Breaking the Statement

Statement: `System.out.println("Hello");`

`System` → Class

`out` → Object (static)

`println()` → Method

Analogy

System → Company

out → Department

println → Employee printing output

Output Statements

- `print()` → stays on same line
- `println()` → moves to next line
- `printf()` → formatted output (%d, %f, %s)

```
public class test {  
    public static void main(String[] args) {  
        // print()  
        System.out.print("Hello ");  
        System.out.print("World");  
        System.out.println(); // line break  
        // println()  
        System.out.println("Welcome to Java Lab");  
        System.out.println("This is println example");  
        // printf()  
        int marks = 85;  
        System.out.printf("Marks = %d", marks);  
    }  
}
```

Other Members of System Class

`System.in` → Standard input

`System.out` → Standard output

`System.err` → Error output

`System.exit()` → Terminates program